

MANUAL DE DESPLIEGUE DEL BACKEND

Spring Boot + Docker Desktop + PostgreSQL en Windows

Este manual describe el procedimiento para desplegar un backend desarrollado con Spring Boot dentro de Docker Desktop, utilizando PostgreSQL instalado directamente en el servidor Windows.

Escenario: Windows 11 + Docker Desktop + Spring Boot (.jar) + PostgreSQL.

1. Arquitectura del despliegue

El backend Spring Boot se ejecuta dentro de un contenedor Docker. PostgreSQL permanece instalado en Windows. El puerto 8080 del contenedor se publica como puerto 8080 del servidor.

```
SERVIDOR WINDOWS
|
+-- Docker Desktop
|   +-- seguimiento_backend
|       +-- Spring Boot :8080
|           |
|           +--> host.docker.internal:5432
|
+-- PostgreSQL :5432

Acceso desde la red:
http://IP_DEL_SERVIDOR:8080
```

2. Requisitos previos

Antes de iniciar, verificar que Docker Desktop esté instalado y muestre "Engine running", que la virtualización esté habilitada, que PostgreSQL esté iniciado y que se disponga del proyecto Spring Boot y de Maven o Maven Wrapper.

3. Configuración de application.properties

El archivo normalmente se encuentra en src/main/resources/application.properties. Cuando Spring Boot se ejecuta directamente en Windows puede conectarse a PostgreSQL mediante localhost. Dentro de Docker, localhost representa al propio contenedor y no al equipo Windows.

```
# Configuración incorrecta dentro de Docker
spring.datasource.url=jdbc:postgresql://localhost:5432/seguimiento_egresado

# Configuración correcta si PostgreSQL está en Windows
spring.datasource.url=jdbc:postgresql://host.docker.internal:5432/seguimiento_egresado
spring.datasource.username=postgres
spring.datasource.password=TU_PASSWORD
spring.datasource.driver-class-name=org.postgresql.Driver

server.port=8080
```

4. Generar el archivo JAR

Abrir PowerShell o una terminal en la carpeta raíz del proyecto y compilar la aplicación.

```
cd C:\proyectos\seguimiento_egresado_backend

mvn clean package

# Si se utiliza Maven Wrapper:
.\mvnw clean package
```

5. Verificar el JAR

Si la compilación finaliza correctamente debe mostrarse BUILD SUCCESS. El archivo generado normalmente estará dentro de la carpeta target.

```
target\seguimiento-egresado-0.0.1-SNAPSHOT.jar
```

6. Crear el Dockerfile

Crear un archivo llamado Dockerfile, sin extensión, en la raíz del proyecto. Ajustar el nombre del JAR al nombre real generado por Maven.

```
FROM eclipse-temurin:21-jre
```

```
WORKDIR /app

COPY target/seguimiento-egresado-0.0.1-SNAPSHOT.jar app.jar

EXPOSE 8080

ENTRYPOINT ["java", "-jar", "app.jar"]
```

7. Construir la imagen Docker

Desde la carpeta donde se encuentra el Dockerfile, construir la imagen. El punto final es obligatorio porque representa el contexto de construcción actual.

```
docker build -t seguimiento_egresado_backend .

docker images
```

8. Crear y ejecutar el contenedor

Crear el contenedor publicando el puerto 8080 del servidor hacia el puerto 8080 del contenedor.

```
docker run -d -p 8080:8080 --name seguimiento_backend seguimiento_egresado_backend
```

9. Verificar el estado

Comprobar que el contenedor permanezca en ejecución y revisar los logs de Spring Boot.

```
docker ps

docker ps -a

docker logs seguimiento_backend

# Logs en tiempo real
docker logs -f seguimiento_backend
```

10. Probar el backend

Desde el propio servidor puede utilizarse localhost. Desde otra computadora de la red se debe utilizar la IP del servidor Windows.

```
# Desde el servidor
http://localhost:8080

# Desde otra PC de la red
http://IP_DEL_SERVIDOR:8080

# Ejemplo
http://172.20.14.250:8080
```

11. Error: Connection to localhost:5432 refused

Este error significa que Spring Boot está intentando encontrar PostgreSQL dentro del mismo contenedor. Si PostgreSQL está instalado en Windows, sustituir localhost por host.docker.internal en application.properties, volver a generar el JAR y reconstruir la imagen.

```
spring.datasource.url=jdbc:postgresql://host.docker.internal:5432/seguimiento_egresado
```

12. El contenedor inicia y se detiene

Si al presionar Play en Docker Desktop el contenedor se detiene inmediatamente, significa que el proceso principal ha fallado. Revisar primero los logs en lugar de crear contenedores adicionales.

```
docker ps -a

docker logs seguimiento_backend
```

```
# Iniciar mostrando directamente la salida
docker start -a seguimiento_backend
```

13. Comandos para administrar el contenedor

Los siguientes comandos permiten detener, iniciar, reiniciar y eliminar el backend desplegado.

```
docker stop seguimiento_backend
docker start seguimiento_backend
docker restart seguimiento_backend
docker rm seguimiento_backend

# Eliminar forzosamente
docker rm -f seguimiento_backend
```

14. Actualización del backend

Cada vez que se modifique el código fuente, debe generarse un nuevo JAR y reconstruirse la imagen. El contenedor existente no se actualiza automáticamente.

```
# 1. Generar nuevo JAR
mvn clean package

# 2. Detener contenedor
docker stop seguimiento_backend

# 3. Eliminar contenedor
docker rm seguimiento_backend

# 4. Reconstruir imagen
docker build -t seguimiento_egresado_backend .

# 5. Crear nuevo contenedor
docker run -d -p 8080:8080 --name seguimiento_backend seguimiento_egresado_backend

# 6. Verificar
docker ps
docker logs seguimiento_backend
```

15. Flujo resumido

El procedimiento completo de actualización y despliegue puede resumirse de la siguiente manera.

```
Modificar application.properties / codigo
|
v
mvn clean package
|
v
Generar nuevo .jar
|
v
docker build
|
v
docker run
|
v
Spring Boot dentro de Docker
|
v
host.docker.internal:5432
|
v
PostgreSQL en Windows
```

Resultado final

Al completar el procedimiento, el backend Spring Boot queda aislado y ejecutándose dentro de Docker, publicado en el puerto 8080, mientras PostgreSQL puede permanecer instalado directamente en Windows. La comunicación entre el contenedor y la base de datos se realiza mediante `host.docker.internal`.